Unit 16 – Procedural Programming
Assignment 1 - Features of Procedural Programming

Introduction

You are working as a trainee for a development company. Your boss wants to be sure that you understand the fundamentals of programming before she lets you loose on an important project. To put her mind at rest, she has asked you to do the following tasks:

(P1) Explain the key features of procedural programs

Explain what a variable is:
In procedural programs, a variable is basically an area of storage of information which can be referenced or manipulated in a computer program, basically containers which hold information. Variables also give a way of labeling data with a specific name, so the programs could be understood more clearly by the reader and the programmer. However, their sole purpose is to label and store data in memory which can be used throughout a program.
Explain the difference between a variable declared in a module (global) and a variable declared in a method (local):
Global variables are variables which can be accessed from anywhere within a program, but global variables generally are variables that do not change. For example, if there's a program which relies in pi it maw use a global variable. On the other hand, local variables are variables that are used in a single function or method, which allows programmers to reuse variable names. For example, the variable ‘i’ is often used for iteration and since it is local, it allows it to be used whenever it is necessary.
Explain what conditional (if/else) logic is and why a programmer might want to use it:
Conditional logic is a feature which can perform different actions depending on whether a boolean condition is evaluated to true or false. For example, if the boolean condition is true it would use the If (consequent) statement, if it is false it would use the Else (alternative) statement. There are many particular tasks that a programmer would use looping for. For example, if a programmer develops a software module for payroll processing that needs to compute the salaries and the allowances of all the employees belonging to a particular cost center, he needs to define the loop accordingly.
Explain what a do loop is and why might a programmer want to use one:
A Do Loop is a loop which it repeats the enclosed block of statements while a Boolean condition is True or until the condition becomes True. A Do Loop can be terminated at any time with the Exit Do statement.

A programmer might use this when he is making a sequence repeat until it reaches a certain point like multiplying or counting numbers, or if the programmer asked a closed question in the module and it required a specific answer.

Explain what a breakpoint is and why a programmer might want to use one:

A breakpoint is a tool which is used to stop the program at a particular place. When using the breakpoint, you click the margins of the code to add one which would cause a red circle to appear, indication where the code will break. When you run the program, you return to the coding window to the place where you put the breakpoint and it will have a yellow arrow on top of the red circle and the line of code will be highlighted yellow.

A programmer might use this if he is 'debugging' the code. This is useful because it makes problem solving a lot more easier due to the fact that you can see where in the code there is a mistake when it is being run, pinpointing where the error/bug on the code is.

Explain what a method is (also known as a function):

A method (or function depending on which programming language is used) is a procedure which performs a task when called, such as data (i.e. an object). Data are properties which are represented in an object and methods as behaviour. For example, the current state of the window would be a property while the object would have methods such as opening and closing.

Explain the difference between a pre-defined (or built-in) method and one created inside a programmer's own program:

Pre-defined (or built-in) methods are methods which have already been created on the system or before the programmer started his creation, which is very beneficial to functionality of the program and the simplicity due to the less amount of repeated code.

However, methods created inside a programmer's own program is an original creation by the programmer, of course, but this could help the programmer due to lack of code which he needs in his program or that the programmer feels that he could improve functionality and simplicity of code.

Explain what it means to "Pass a parameter into a method":

When you pass a parameter into a method, you are basically telling the method what to do. For example, if you type in a parameter (in this case its 'enter your name') into a method, the method would be written as 'Console.WriteLine', which means that the console will display on the screen what you have written in the parameter, and in this case it would show up as 'enter your name'.

Explain what it means to "Return a value from a method":

To return a value from a method, you are using the method declaration to declare a method's return type. What I mean by this is that within the body of the method, you use a return statement to return the value. However, it's not required to contain a return statement, but it

can be used to exit the method or the control block. If a method is declared as 'void', it means that it didn't return a value.
Explain what a modular program is:
In modular programming, there are separate sub-programs called modules which has been divided. A module can be used in a variety of applications and functions with other components of the system, with similar functions grouped in the same unit of programming code and separate functions are developed as separate units of code so that the code can be reused by other applications. The modular programming concept is compatible with OOP (Object-oriented programming) to a large extent. Modular programming is helpful because it enables multiple programmers to divide up the work and debug pieces of the program independently
Explain what a programming library is:
A programming library is basically a file which contains pre-defined code that could be useful within the program. The code could simply be a function or a collection of functions, classes, variable and data structures. In simple terms, a library in programming is a lot like a book library, but instead of a physical collection of books, it's a collection of code which is already written (hence why it's pre-defined code).

(P2) Demonstrate the use of procedural tools and techniques

Include the code from any .NET VB project that you have created in which you have demonstrated use of the following skills:

Storing information in a variable

We know that variables are basically containers for information which can be manipulated and referenced in the program. The first way I will mention on how to store information in a variable is the integer variable.

This variable basically stores numbers as data in the variable which can be used for anything that the programer wants to do with it (preferably math). In the code below, basically shows a simple '3 + 5' equation made in visual basic.

There are 3 integer variables, and the variables have been set up, which are: 'number1', 'number2' and 'answer'. Then, two of the integer variables are used to contain the data, the first one (or 'number1') containing '3', and the second one (or 'number2') containing '5'. The way this was done is that the data was added into the variable using the equals (=) sign, but it essentially isn't an equals sign, it's considered as an assignment operator.

The next part is slightly more complicated, but in theory it's not complicated at all. What was done is that both the numbers were added together by saying ‘number1 = 3’ ; ‘number 2 = 5’.

```
Dim number1 As Integer
Dim number2 As Integer
Dim answer As Integer

number1 = 3
number2 = 5

answer = number1 + number2
```

Now that I’ve covered what a variable is, how to set one up and about the word ‘integer’, and that integer variables hold numbers as data. However, variables also have another way of storing data that isn't numbers, this is where Strings come in.

In very basic terms, a string is nothing more than text, and if we want Visual Basic to store text we just use the word ‘String’. For example, the code below, you can see that it started with the Dim word, just like the previous example.

The first variable is named ‘FirstName’ and finally it ends with the line telling Visual Basic that the text is being stored in the variable as a String. This is repeated on the 2nd and 3rd one but the only difference is the text being ‘LastName’ and ‘FullName’.

After the variables have been set up, there still isn't anything stored in them yet, that’s when the assignment operator (which is basically an equals sign(=)) is used to store text, in the code below its ‘Bill’ and ‘Gates’. This stores ‘Bill’ into the FirstName string variable and ‘Gates’ into the LastName string variable.

Then the two variables are added together, this is done using the ampersand symbol (&) instead of the plus (+) symbol used in the integer variables. Both the ampersand and plus does the same thing, the only difference is that the plus is used on the integer variables and the ampersand is used on string variables.

```
Dim FirstName As String
Dim LastName As String
Dim FullName As String

FirstName = “Bill”
LastName = “Gates”

FullName = FirstName & LastName
```

Retrieving information from a variable (e.g. passing a variable into a sub or function for use as a parameter, or using in a calculation).

In this code, the program is retrieving the information from the variables, the variables being ‘name’ (as a String variable), ‘age’ (as a Integer variable) and ‘heightInMetres’ (as a Double variable). The information in the variables aren’t written down in the program. Instead, the data that the variables would collect is determined to what the user would type into the console application.

```
Dim name As String
Dim age As Integer
Dim heightInMetres As Double

Console.WriteLine("Welcome ...")

Console.Write("What is your name? : ")
name = Console.ReadLine()

Console.Write("What is your age? : ")
age = Convert.ToInt32(Console.ReadLine())

Console.Write("What is your height in metres? : ")
heightInMetres = Console.ReadLine()
```

When the user types in something inside the console application, he would get a message from the console saying ‘welcome…’ and then he would be asked, ‘What is your name?’. For example, if the user types in the name ‘Leo’, the console would gather that information using the ‘Console.ReadLine()’ and store it into the variable.

Using a method that is a part of the Console programming library (e.g. WriteLine()).

In this code, there are 2 methods that are a part of the Console programming library. The two parts are the ‘WriteLine’ and the second one being ‘Readline’. Both of these functions do different jobs, even though the only difference between the two is the ‘Read’ and ‘Write’.

```
Console.WriteLine("Enter a name to remove from list: ")
name = Console.ReadLine()
```

The ‘WriteLine’ method is a method in which it writes the current line terminator to the standard output stream. What I mean by this is that the program basically writes the parameter that was typed into the console application.

The ‘ReadLine’ method is a method which reads the next line of characters from the standard input stream. What I mean by that is that the program basically reads the user input written, but only if the user has written the line of characters after it reaches the point of the ‘ReadLine’ in the code.

Passing data as a parameter into a method (e.g. WriteLine()).

In the code below, there is a method called ‘WriteLine’. As I stated before, the ‘WriteLine’ is what the program would write down in the console when it reaches the ‘WriteLine’ code after the launch of the console application.

```
Console.WriteLine("Welcome ...")
```

In the code above, the ‘WriteLine’ method has ‘Welcome ... ‘ written as a parameter. When the console application is launched, the console will write the parameter in the console application when it reaches that part of the code.

Using information returned from a method (e.g. ReadLine()) returning information into a variable

In the code below, there are 3 different ‘WriteLine’ code that write ‘Hello’, ‘You are … Years old’ and ‘You are … metres tall’.

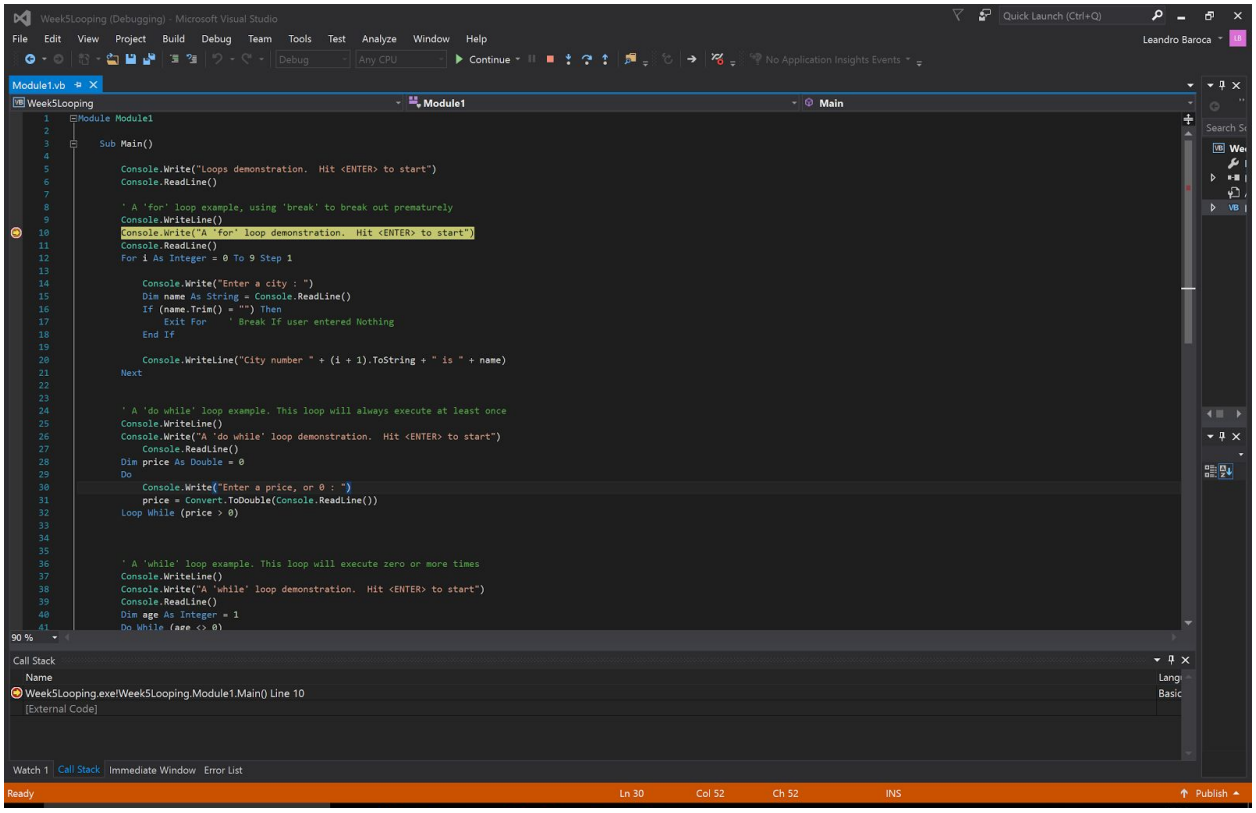
```
Console.WriteLine("Hello " + name)
Console.WriteLine("You are " + age.ToString() + " years old")
Console.WriteLine("You are " + heightInMetres.ToString() + " metres tall")
```

When the console application is launched, and the user has done the previous commands that was written by the program into the console. The user will eventually reach this stage of the code, which would gather the information that was gathered before and write it down the the console application.

For example, the user may have written the name ‘Leo’, written the age ‘17’ and written the height as ‘189’. The program would of stored this information in the variables: ‘name’, ‘age’ and ‘heightInMetres’. The program would then gather the stored information in the variables and write it in the console application as a .ToString.

Provide a screen-print that shows:

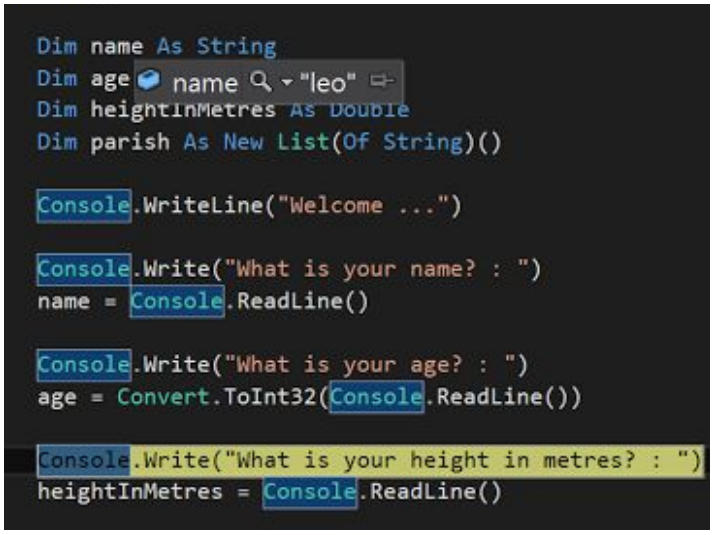
A program that you have written stopped at a break-point



In the screen-print above, I made a breakpoint which is basically a tool which can be set anywhere in the code and that code will execute to that point and then will stop. The breakpoints can be set when writing the code, or it can be set at runtime by switching to the code and setting the breakpoint at a desired location.

When Visual Studio encounters a breakpoint, the execution of the code stops and then the program will get into a ‘break mode’. During ‘break mode’, you can use a lot of debug features which become available during the break mode.

The content of a variable that has some information stored in it.



In the screen-print above, I opened a different Visual Basic project because it would be a better example to use than the one I previously took a screen-print of, and that I feel this has much better variables to store information in.

In the code above, I used the breakpoint at the end of the 3rd 'Console.Write' code so that I could use the previous 2 to store the information for the variable. As you can see, when I hovered my mouse over the 'name' variable, it came up with the stored information in the variable. In this case it was 'Leo'.

If I hovered my mouse over the age variable it would of came up with the number '17' because that's what I had written down in the console application, which stored it in the age variable as an integer variable (due to the fact the information consisted of numbers).

This is good for checking for errors or debugging it because you can see whether there is missing information within a variable, and fix the error faster than not using the debugger at all. Not using the debugger could be time consuming at a significant rate.

(M1) Explain why modular elements are important for procedural programming

How creating your own subs/functions simplifies the code that makes use of these subs/functions

The reason when you create your own methods and how it simplifies the code is that you can reduce the amount of code in the program because you can add different, general methods and also using different codes which can be referred or its name being qualified (also known as scopes).

This method also gives the programmer less things to remember or search up because he is creating his own subs and functions, which would decrease the amount of time needed to create the program. This would prove to be very important if there is a deadline for the program or there is a very short amount of time.

Whe re-using the same subs/functions reduces the total amount of coding required and the amount of testing required

The reason less coding is required when re-using the same sub/functions is that the programmer could reference the subs or functions wherever he wants within the program, instead of repeating the code over and over with different subs and functions that are the exact same.

The reason that this is very important is that a lot of cluttered code would slow down a PC (depending on its processor), especially with a lot of unnecessary repeated code. This is also why it improves the testing of the program, because with less repeated and cluttered code it would take longer for the system to run the program for a test.

How a language that allows modules to be split into different files enables teams of programmers to work together effectively

Modular programming is very useful for teams of programmers due to fact that it parts of the program can be split into different sub-programs which can be used by individual programmers. The more programmers working on one program, the faster the program will be complete.

However, it does not necessarily mean that the program will be more well made or efficient, but it is possible. It is debatable that 1 programmer would make his program more efficient than a team, but it depends on who the individual programmer is and also who is in the team and how organised the team is.

Why a programmer might want to include modules of code found in libraries written by other people in their programs

Using code found in libraries written by other people can prove to be very beneficial, and there are a few reasons for this. For example, there might be a programmer with little to no experience and if he is using code made by others which he wouldnt of come up himself or needed in his program it would benefit him a lot.

However, this does not only apply to inexperienced programmers, sometimes programmers even with a lot of experience may sometimes need the code written by others because it's less time consuming or that other people’s code are much more creative than he would of made himself.

(D1) Evaluate the suitability of procedural programs for graphical applications

Written report of approximately 500 to 1000 that:

Describes two different graphical applications (not games) that make use of graphical elements (e.g. drop down boxes, radio buttons, checkboxes, text boxes, pictures or animation).

The first application I will be mentioning is the Microsoft Calculator application. The application is a very simplistic calculator that could be used as an alternative to a different calculator software or to a physical calculator. There isn't much to say about the purpose of the application itself but it does have a few graphical elements.

For example, it has its own drop down box which have a variety of setting for the calculator, such as a ‘standard’, ‘scientific’, ‘programmer’ and ‘Date calculation’. It also has many different options for ‘converter’ which has ‘volume’, ‘length’, etc.

Other examples are the option to make the calculator application maximise and restore the size by clicking the maximise/restore button on the top right of the application. The buttons can also be pressed using the mouse cursor and they are even ‘touch-friendly’ (if you have a touchscreen monitor). The buttons on the app also have a very simple animation of the colour of the button lighting up/darkening (depending whether you have a dark theme or a light theme) when pressed on.

The second application I will be mentioning is the Scratch application. Scratch is basically a very basic, somewhat minimalistic (compared to actually programming languages) and ‘modern’ looking ‘programming’ application. Apart from the simplistic and modern graphical design of the app, the app also includes different graphical elements to it.

For example, it has some graphical features which make the ‘coding’ in the app very easy to use. What I mean by this is that each fragments of code act a lot like jigsaw puzzles which can interlock with each other, which allows the user to see which code has been added or fitted to another code.

Explains why the graphical elements are useful in these applications

The graphical elements on the calculator prove to be useful because it makes the calculator very versatile due to the different calculator and converter options on the drop down box. If no drop down box was provided on this application it would be a lot more difficult to make the calculator app as versatile as it is now.

The ability to maximize/restore the size gives the flexibility to use the app to be divided or to cover the whole screen to the user preferability and ease of use. The lighting up of buttons also can be useful due to the fact that it shows the button you are clicking on, it may not be so useful

with a mouse cursor because you can see where it is, but with a touchscreen monitor it isn't as easy to predict what button you have pressed.

The graphical elements on the Scratch application prove useful because, as I mentioned before, the 'jigsaw puzzle' type interface makes it very easy for the user, especially if it's a child or someone new to programming in general.

For example, this type of interface makes it less intimidating for a child or someone brand new to programming when compared to a different programming language such as C++, Java and other heavily used programming languages.

Explains how a replacement procedural console application might differ from the graphical applications

If the calculator application's purpose was in a procedural console application, it would instantly remove the simplistic and modern graphical look of the application. Another difference is that it wouldn't be nearly as versatile as Microsoft's calculator because it would be impossible to include a dropdown box with all the different calculator and converter options which may come in handy depending on the user's needs for the calculator app.

However, if you are using the calculator for it's simple purpose: calculating addition, subtraction multiplication and division. It wouldn't be much different because it is possible to create a calculator in a programming language such as Visual Basic.

If Scratch was replaced for a procedural console application, it would instantly remove the modern and simplistic graphical interface of the program, because it's now in a console application. The 'jigsaw puzzle' interface would also be non existent and the user wouldn't be able to see the physical program with the avatars being controlled by the code that was placed down.

Discusses how this would affect the overall usefulness and desirability of the applications

With the Calculator application, as a console application it wouldn't make much difference if it was the most basic use of a calculator, because it's possible in a console based programming language such as Visual Basic to create one. However, the sleek and modern look of the calculator application is non existent, which may be undesirable by some people.

If someone was still desired to use the console app, they wouldn't have the versatility of the calculator app, which removed a huge chunk of usefulness from the app because it doesn't stretch its usefulness as far as the versatile one.

With Scratch, as a console application it would instantly remove the purpose the application was made: make coding easy to do. The reason for this is due to the fact that Scratch's purpose is to

be a coding application less intimidating than traditional coding and making it less easier for people who never coded before and a lot less child friendly. So, a console application of Scratch would be pointless for its actual purpose.